

Mudcard

- **the class itself was clear enough but I would appreciate a bit more info about the group project and maybe a bit more instructions on that**
 - Yep, the announcement is planned for Friday.
- **I did not really get the "bins". What does it do?**
 - It changes how many bins the histogram has.
 - Please read the manual and play around with the code.
- **I wouldn't say anything was muddy but I could use help between differentiating between when to use different visualizations**
 - It's not easy because there are so many options.
 - For a single feature, the decision is pretty easy. Histogram for a continuous feature, and bar plot for categorical or ordinal features.
 - The complication of ordinal features is that the categories need to be ordered which matplotlib doesn't do by default
 - For two columns, there is a handy 2x2 matrix in the lectures notes to help you decide
 - There are also a bunch of additional resources and websites at the end of lecture note.
- **I was just confused by quiz 2 (which you can probably see), but I think I get the logic for all of them now.**
 - Nice! I'm glad to hear the quiz helps with understanding, that's exactly the goal!

Lecture 5: Data splitting, part 1

Split iid data

By the end of this lecture, you will be able to

- describe what the iid assumption is
- apply basic split to iid datasets
- apply k-fold split to iid datasets

The supervised ML pipeline

0. Data collection/manipulation: you might have multiple data sources and/or you might have more data than you need

- you need to be able to read in datasets from various sources (like csv, excel, SQL, parquet, etc)
- you need to be able to filter the columns/rows you need for your ML model
- you need to be able to combine the datasets into one dataframe

1. Exploratory Data Analysis (EDA): you need to understand your data and verify that it doesn't contain errors

- do as much EDA as you can!

2. Split the data into different sets: most often the sets are train, validation, and test (or holdout)

- practitioners often make errors in this step!
- you can split the data randomly, based on groups, based on time, or any other non-standard way if necessary to answer your ML question

3. Preprocess the data: ML models only work if X and Y are numbers! Some ML models additionally require each feature to have 0 mean and 1 standard deviation (standardized features)

- often the original features you get contain strings (for example a gender feature would contain 'male', 'female', 'non-binary', 'unknown') which needs to be transformed into numbers
- often the features are not standardized (e.g., age is between 0 and 100) but it needs to be standardized

4. Choose an evaluation metric: depends on the priorities of the stakeholders

- often requires quite a bit of thinking and ethical considerations

5. Choose one or more ML techniques: it is highly recommended that you try multiple models

- start with simple models like linear or logistic regression
- try also more complex models like nearest neighbors, support vector machines, random forest, etc.

6. Tune the hyperparameters of your ML models (aka cross-validation or hyperparameter tuning)

- ML techniques have hyperparameters that you need to optimize to achieve best performance
- for each ML model, decide which parameters to tune and what values to try
- loop through each parameter combination
 - train one model for each parameter combination
 - evaluate how well the model performs on the validation set
- take the parameter combo that gives the best validation score
- evaluate that model on the test set to report how well the model is expected to perform on previously unseen data

7. Interpret your model: black boxes are often not useful

- check if your model uses features that make sense (excellent tool for debugging)
- often model predictions are not enough, you need to be able to explain how the model arrived to a particular prediction (e.g., in health care)

Split iid data

By the end of this lecture, you will be able to

- describe what the iid assumption is
- apply basic split to iid datasets
- apply k-fold split to iid datasets

Let's revisit the papaya example from the first lecture!

- **the learner's input:**
 - Domain set $\mathcal{X}$ - a set of objects we wish to label (*all papayas on the island*).
 - The probability distribution over $\mathcal{X}$ is $\mathcal{D}$.
 - $\mathcal{D}$ is pretty general
 - $\mathcal{D}$ might change as you sample from it
 - there might be multiple distributions you sample from
 - one sample might depend on one or multiple previous samples, etc.
 - Label set $\mathcal{Y}$ - a set of possible labels (*a papaya can be either tasty or not tasty*).
 - There is some correct labeling function $f : \mathcal{X} \rightarrow \mathcal{Y}$.
 - **a training example is then generated by sampling x_i from $\mathcal{D}$, and the label y_i is generated using f .**
 - Training data $S = ((x_1, y_1), \dots, (x_m, y_m))$ - a finite sequence of pairs from $\mathcal{X}$, $\mathcal{Y}$. This is what the learner has access to (*the data I collected by sampling some papayas*).
 - $X = (x_1, \dots, x_m)$ is the feature matrix which is usually a 2D matrix, and $Y = (y_1, \dots, y_m)$ is the target variable which is a vector.

I.I.D. assumption

- **the i.i.d. assumption:** the examples in the training set are independently and identically distributed according to $\mathcal{D}$
 - every x_i is freshly sampled from $\mathcal{D}$ and then labelled by f
 - that is, x_i and y_i are picked independently of the other instances
 - S is a window through which the learner gets partial info about $\mathcal{D}$ and the labeling function f
 - the larger the sample gets, the more likely it is that $\mathcal{D}$ and f are accurately reflected
- examples of not iid data:
 - data generated by time-dependent processes
 - data has group structure (samples collected from e.g., different subjects, experiments, measurement devices)

- sampling from the distribution changes the properties of the distribution
- we will get back to this later in the term

Quiz 1

Which of these data generation processes or ML problems are not iid?

Split iid data

By the end of this lecture, you will be able to

- describe what the iid assumption is
- **apply basic split to iid datasets**
- apply k-fold split to iid datasets

Why do we split the data?

- we want to find the best hyper-parameters of our ML algorithms
 - fit models to training data
 - evaluate each model on validation set
 - we find hyper-parameter values that optimize the validation score
- we want to know how the model will perform on previously unseen data
 - apply our final model on the test set

We need to split the data into three parts!

Splitting strategies for iid data: basic approach

- 60% train, 20% validation, 20% test for small datasets
- 98% train, 1% validation, 1% test for large datasets
 - if you have 1 million points, you still have 10000 points in validation and test which is plenty to assess model performance

Let's work with the adult data!

```
In [ ]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

df = pd.read_csv('../data/adult_data.csv')

# let's separate the feature matrix X, and target variable y
y = df['gross-income'] # remember, we want to predict who earns more than 50k or less than 50k
X = df.loc[:, df.columns != 'gross-income'] # all other columns are features
print(y)
print(X.head())
```

```
In [ ]: # all sklearn transformers and models accept polars dataframes!
help(train_test_split)
```

```
In [ ]: random_state = 137

# first split to separate out the training set
X_train, X_other, y_train, y_other = train_test_split(X, y, \
                                                    train_size = 0.6, random_state = random_state)
print('training set:', X_train.shape, y_train.shape) # 60% of points are in train
print(X_other.shape, y_other.shape) # 40% of points are in other

# second split to separate out the validation and test sets
X_val, X_test, y_val, y_test = train_test_split(X_other, y_other, \
                                                train_size = 0.5, random_state = random_state)
```

```
print('validation set:',X_val.shape, y_val.shape) # 20% of points are in validation
print('test set:',X_test.shape, y_test.shape) # 20% of points are in test

print(X_train.head())
```

Randomness due to splitting

- the model performance, validation and test scores will change depending on which points are in train, val, test
 - inherent randomness or uncertainty of the ML pipeline
- change the random state a couple of times and repeat the whole ML pipeline to assess how much the random splitting affects your test score
 - you would expect a similar uncertainty when the model is deployed

Quiz 2

What's the second train_test_split line if you want to end up with 60-20-20 in train-val-test? Print out the sizes of X_train, X_val, X_test to verify!

```
In [ ]: X_other, X_test, y_other, y_test = train_test_split(X,y,\
               train_size = 0.8,random_state=random_state)
# add your line below and choose the correct solution from canvas
```

Split iid data

By the end of this lecture, you will be able to

- describe what the iid assumption is
- apply basic split to iid datasets
- apply k-fold split to iid datasets**

Other splitting strategy for iid data: k-fold splitting

 No description has been provided for this image

```
In [ ]: from sklearn.model_selection import KFold
help(KFold)
```

```
In [ ]: random_state =42

# first split to separate out the test set
X_other, X_test, y_other, y_test = train_test_split(X,y,test_size = 0.2,random_state=random_state)
print(X_other.shape,y_other.shape)
print('test set:',X_test.shape,y_test.shape)

# do KFold split on other
kf = KFold(n_splits=5,shuffle=True,random_state=random_state)
for train_index, val_index in kf.split(X_other,y_other):
    X_train = X_other.iloc[train_index]
    y_train = y_other.iloc[train_index]
    X_val = X_other.iloc[val_index]
    y_val = y_other.iloc[val_index]
    print('  training set:',X_train.shape, y_train.shape)
    print('  validation set:',X_val.shape, y_val.shape)
    # the validation set contains different points in each iteration
    print(X_val[['age','workclass','education']].head())
```

How many splits should I create?

- tough question, 3-5 is most common
- if you do n splits, n models will be trained, so the larger the n, the most computationally intensive it will be to train the models

- KFold is usually better suited to small datasets
- KFold is good to estimate uncertainty due to random splitting of train and val, but it is not perfect
 - the test set remains the same

Why shuffling iid data is important?

- by default, data is not shuffled by Kfold which can introduce errors!

 No description has been provided for this image

Mudcard

In []: